

PLAN DE TAREAS DE DESARROLLO

Distribución 360 — ERP Multitenant

Marzo 2026 | Para ejecución en Cursor AI

Instrucciones para Cursor

Este documento contiene todas las tareas técnicas del proyecto organizadas por prioridad. Cada tarea incluye los archivos exactos a modificar, los pasos concretos a ejecutar y notas de contexto. Ejecutar las tareas en el orden indicado (P0 → P1 → P2 → P3) para evitar dependencias rotas.

Prioridad	Descripción
 P0 — CRÍTICO	Bloqueos funcionales o datos en estado inconsistente. Ejecutar primero.
 P1 — ALTO	Brechas de seguridad o flujos core sin implementar. Ejecutar en segunda pasada.
 P2 — MEDIO	Deuda técnica y organización del código. Ejecutar en tercera pasada.
 P3 — BAJO	Mejoras de calidad y documentación. Ejecutar cuando haya ventana de tiempo.

P0 — Tareas Críticas

Estos problemas producen inconsistencias de datos, errores silenciosos en producción o hacen que el sistema muestre información incorrecta al usuario. Deben resolverse antes de cualquier otra cosa.

T1 | Conectar el Dashboard con datos reales

PROBLEMA: La página principal del dashboard muestra KPIs hardcoded (\$45,231.89, +573 pedidos). El usuario ve datos ficticios como si fueran reales.

Backend — Crear endpoint de estadísticas

- Crear archivo: apps/backend/src/dashboard/dashboard.controller.ts
- Crear archivo: apps/backend/src/dashboard/dashboard.service.ts
- El endpoint GET /dashboard/company/:companyId/stats debe retornar:

```
{ totalSalesMonth: number, activeOrders: number, totalArticles: number, lowStockCount: number }
```
- Registrar DashboardModule en apps/backend/src/app.module.ts

Frontend — Consumir el endpoint

- Editar: apps/web/app/dashboard/[id]/page.tsx
- Reemplazar los valores hardcoded por una llamada a getCompany() + fetch al nuevo endpoint
- Agregar función getDashboardStats(companyId) en apps/web/lib/api.ts
- Mostrar un skeleton loader mientras cargan los datos

⚠ *Nota: Si los módulos de ventas aún no existen, el endpoint puede retornar ceros reales en lugar de datos ficticios.*

T2 | Corregir inconsistencia search_vector en variantes

PROBLEMA: La migración ArticlesHighPriorityImprovements eliminó la columna search_vector y su índice GIN, pero el trigger article_variants_search_trigger en sql1.txt la sigue referenciando. El buscador del inventario y del POS puede estar fallando silenciosamente.

- Verificar en Supabase si la columna search_vector existe en article_variants
- Opción A — Si no existe: eliminar o comentar el trigger article_variants_search_trigger en apps/database/sql1.txt
- Opción B — Si se quiere recuperar el buscador full-text: crear nueva migración que re-añada search_vector como columna generada con los pesos correctos
- Crear migración: apps/backend/src/migrations/[timestamp]-FixSearchVector.ts
- Probar el endpoint GET /articles/company/:id/search-variants?q=test y verificar que retorna resultados

⚠ *Nota: Si se elige Opción B, el trigger debe usar BEFORE INSERT OR UPDATE y NEW.search_vector debe asignarse dentro de la función PL/pgSQL.*

T3 | Envolver operaciones de barcodes en transacción

PROBLEMA: En createVariant y updateVariant, los barcodes adicionales se guardan con saves individuales dentro de un for-loop sin QueryRunner. Si el tercer barcode falla, los dos primeros ya están guardados y el artículo queda en estado inconsistente.

- Editar: apps/backend/src/articles/articles.service.ts
- En el método createVariant: obtener un QueryRunner desde `this.dataSource.createQueryRunner()`
- Envolver toda la operación (save variant + save price + save barcodes) en `queryRunner.startTransaction()`
- Usar try/catch: en el catch hacer `queryRunner.rollbackTransaction()`, en el finally `queryRunner.release()`
- Repetir el mismo patrón en el método updateVariant para la sección de barcodes
- Inyectar DataSource en el constructor: `constructor(private dataSource: DataSource, ...)`

⚠ Nota: TypeORM requiere importar DataSource desde 'typeorm'. No usar EntityManager directamente.

P1 — Tareas de Alta Prioridad

Brechas de seguridad activas y flujos de negocio core que el sistema presenta como disponibles pero no están implementados.

T4 | Implementar RoleGuard en endpoints sensibles

PROBLEMA: Todos los endpoints de inventario, contactos y configuración solo validan que el usuario tenga un JWT válido (JwtAuthGuard) y que el módulo esté activo (ModuleEnabledGuard). No verifican si el ROL del usuario tiene permiso de edit, delete o annull. Cualquier usuario autenticado puede borrar artículos.

- Crear archivo: apps/backend/src/common/guards/role.guard.ts
- El guard debe leer req.user.permissions y verificar que contenga la acción requerida
- Crear decorador: apps/backend/src/common/decorators/require-permission.decorator.ts
`@RequirePermission('inventory', 'delete')`
- Aplicar en apps/backend/src/articles/articles.controller.ts: proteger DELETE y PATCH con el nuevo guard
- Aplicar en apps/backend/src/contacts/contacts.controller.ts: proteger DELETE y PATCH
- Aplicar en apps/backend/src/roles/roles.controller.ts: ya usa AdminGuard, verificar que sea suficiente
- Agregar al menos los permisos: view, edit, delete, annull para los módulos: inventory, contacts, users

⚠ Nota: El SUPER_ADMIN tiene permissions: { '': true } en el JWT, el guard debe hacer bypass si ese wildcard está presente.*

T5 | Implementar flujo completo del Punto de Venta (POS)

PROBLEMA: La página /pos solo tiene el buscador de variantes. El módulo POS está en el menú principal como funcionalidad prioritaria pero no puede completar ninguna venta.

Frontend

- Editar: apps/web/app/dashboard/[id]/pos/page.tsx
- Añadir estado del carrito: [{ variantId, name, qty, price, subtotal }]
- Al seleccionar una variante del buscador, añadirla al carrito con qty=1 y el precio de pvp1
- Mostrar tabla de carrito con columnas: Producto, Cantidad, Precio Unit., Subtotal, Eliminar
- Calcular y mostrar: subtotal, IVA, total
- Añadir campo de búsqueda de cliente (consumidor final por defecto)
- Botón 'Confirmar Venta' que llame al endpoint del backend

Backend

- Crear: apps/backend/src/sales/sales.module.ts

- Crear: apps/backend/src/sales/sales.controller.ts con POST /sales
- Crear: apps/backend/src/sales/sales.service.ts con método create()
- El método create debe: validar stock si preventNegativeStock=true, descontar stock, guardar la venta, retornar el ID
- Crear entidades: Sale, SaleLine en apps/backend/src/sales/entities/
- Registrar SalesModule en apps/backend/src/app.module.ts

⚠ Nota: Para la primera versión no es necesario generar el XML del SRI. La venta puede guardarse como 'borrador' o 'completada' sin comprobante electrónico.

T6 | Implementar Refresh Token en autenticación

PROBLEMA: El JWT expira en 8 horas sin mecanismo de renovación. Al expirar, el usuario recibe errores 401 silenciosos en operaciones de fondo. No hay redirect automático al login.

- Editar: apps/backend/src/auth/auth.service.ts — añadir método refreshToken(oldToken)
- Editar: apps/backend/src/auth/auth.controller.ts — añadir endpoint POST /auth/refresh
- El endpoint debe verificar el token anterior (aunque esté próximo a expirar) y emitir uno nuevo
- Crear: apps/web/app/api/auth/refresh/route.ts — proxy del endpoint de refresh
- Crear: apps/web/lib/auth-interceptor.ts — función que intercepte respuestas 401 y reintente con refresh
- En apps/web/lib/api.ts y api-client.ts, envolver todas las llamadas fetch con el interceptor
- Si el refresh también falla con 401, redirigir al usuario a /login

P2 — Deuda Técnica (Organización del Código)

Estos cambios no afectan la funcionalidad visible pero son críticos para la mantenibilidad a medida que el equipo crece y se añaden más módulos.

T7 | Extraer lógica de precios de `article-form-dialog.tsx`

PROBLEMA: El componente `article-form-dialog.tsx` contiene cientos de líneas con lógica de cálculo de rentabilidad (`handleCostToPriceCalculation`, `refreshRentabilidadOnCostBlur`, `handlePvpCalculation`, etc.) mezclada con el JSX del formulario. Esto hace imposible reutilizar o probar esa lógica.

Paso 1 — Crear el custom hook

- Crear: `apps/web/lib/hooks/usePriceCalculation.ts`
- Mover todas las funciones de cálculo al hook: `handleCostToPriceCalculation`, `refreshRentabilidadOnCostBlur`, `handlePvpCalculation`, `handleSalePriceCalculation`, `applyPctRentBlurOrEnter`, `applyPvpCellBlurOrEnter`
- El hook debe recibir como parámetros: `taxId`, `taxes`, `variants` y un setter `setVariants`
- El hook debe retornar las funciones de cálculo ya vinculadas

Paso 2 — Crear la capa de tipos compartidos

- Crear directorio: `apps/web/lib/types/`
- Crear: `apps/web/lib/types/article.types.ts` con los tipos: `VariantRow`, `PricesRow`, `Batch`, `FractionConfig`, `ArticleImage`, `AdditionalBarcode`
- Exportar todos los tipos desde `apps/web/lib/types/index.ts`
- Actualizar todos los archivos que usen esos tipos para importarlos desde la nueva ubicación

Paso 3 — Limpiar el componente

- En `apps/web/app/dashboard/[id]/inventory/article-form-dialog.tsx`:
- Reemplazar las funciones de cálculo con las importadas desde el hook
- Reemplazar las definiciones de tipos inline con los imports desde `apps/web/lib/types`

⚠ Nota: Hacer este refactor en una rama separada. No combinar con otros cambios de funcionalidad.

T8 | Centralizar `API_BASE` y crear capa de servicios API

PROBLEMA: La constante `API_BASE = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3001'` está duplicada en 5+ archivos (`api.ts`, `api-client.ts`, `pos/page.tsx`, `route handlers de auth`). Un cambio de URL requiere editar múltiples archivos.

- Crear: `apps/web/lib/config.ts` con:

```
export const API_BASE = process.env.NEXT_PUBLIC_API_URL ??
'http://localhost:3001';
```

- Editar apps/web/lib/api.ts — eliminar la definición local de API_BASE, importar desde config.ts
- Editar apps/web/lib/api-client.ts — mismo cambio
- Editar apps/web/app/dashboard/[id]/pos/page.tsx — mismo cambio
- Editar apps/web/app/api/auth/login/route.ts — mismo cambio
- Editar apps/web/app/api/auth/register/route.ts — mismo cambio
- Editar apps/web/app/api/auth/select-company/route.ts — mismo cambio
- Crear: apps/web/lib/services/articles.api.ts con todas las funciones de artículos actualmente en api-client.ts

⚠ *Nota: Verificar que ningún archivo use la constante directamente como string literal antes de eliminarlo.*

T9 | Refactorizar layout del dashboard

PROBLEMA: apps/web/app/dashboard/[id]/layout.tsx contiene condicionales de negocio (isAdminOrOwner, isSuperAdmin, verificaciones de módulo) directamente en el JSX del layout. El archivo es difícil de leer y de modificar.

- Crear: apps/web/components/dashboard/AdminNav.tsx — navegación exclusiva para admin/owner
- Crear: apps/web/components/dashboard/OperationsNav.tsx — navegación de operaciones (POS, Inventario, Directorio)
- Crear: apps/web/components/dashboard/SuperAdminNav.tsx — navegación de super admin
- Editar apps/web/app/dashboard/[id]/layout.tsx — reemplazar el JSX de navegación con los nuevos componentes
- Cada componente de nav recibe como prop las variables de permisos (isAdminOrOwner, moduleStatus, etc.)
- El layout queda como: <SuperAdminNav /> | <OperationsNav /> + <AdminNav /> según el rol

P3 — Mejoras de Calidad y Documentación

Estas tareas mejoran la calidad general del proyecto. No tienen urgencia operativa pero son importantes para escalar el equipo y reducir bugs futuros.

T10 | Añadir pruebas automatizadas a lógica crítica

PROBLEMA: El único test existente verifica que GET / retorna 'Hello World!'. No hay ninguna cobertura de la lógica de negocio del módulo más complejo del sistema.

Backend (Jest)

- Crear: apps/backend/src/articles/articles.service.spec.ts
- Test: createVariant debe lanzar ConflictException si el SKU ya existe en la empresa
- Test: createVariant debe lanzar ConflictException si el barcode ya está asignado
- Test: updateVariant debe sincronizar articleCode desde article.code
- Crear: apps/backend/src/companies/companies.service.spec.ts
- Test: assertResourceLimit debe lanzar ForbiddenException si se supera el límite
- Test: assertResourceLimit no debe lanzar si el usuario es SUPER_ADMIN

Frontend (Vitest o Jest)

- Crear: apps/web/lib/math.util.test.ts
- Test: roundToFive(0.10000000000000001, 5) debe retornar 0.1
- Test: roundToFive(1.23456789, 5) debe retornar 1.23457
- Crear: apps/web/lib/cost-iva.test.ts
- Test: costToCostInclva(100, 15) debe retornar 115
- Test: costInclvaToCost(115, 15) debe retornar 100

T11 | Configurar Swagger / OpenAPI en el backend

PROBLEMA: No hay documentación de la API. El equipo frontend y cualquier integración externa debe leer el código fuente para conocer los contratos.

- Instalar: npm install @nestjs/swagger en apps/backend
- Editar: apps/backend/src/main.ts — inicializar SwaggerModule con DocumentBuilder
- Configurar: título 'Distribución 360 API', versión '1.0', tag por módulo
- Añadir @ApiTags('articles') en apps/backend/src/articles/articles.controller.ts
- Añadir @ApiTags('auth') en apps/backend/src/auth/auth.controller.ts
- Añadir @ApiTags('contacts') en apps/backend/src/contacts/contacts.controller.ts
- Añadir @ApiProperty() en los DTOs más utilizados: CreateArticleVariantDto, RegisterDto, CreateContactDto
- Verificar que /api/docs esté accesible en desarrollo (no exponer en producción sin auth)

T12 | Implementar la página de Auditoría

PROBLEMA: El módulo de Auditoría aparece en el menú y en los planes de suscripción pero no tiene página implementada.

- Crear directorio: `apps/web/app/dashboard/[id]/settings/audit/`
- Crear: `apps/web/app/dashboard/[id]/settings/audit/page.tsx`
- Crear endpoint backend: `GET /audit/company/:companyId?page=1&limit=50`
- El endpoint debe leer de una tabla `audit_logs` (crearla si no existe) con campos: `id`, `company_id`, `user_id`, `action`, `entity`, `entity_id`, `created_at`
- La página debe mostrar una tabla paginada con: fecha, usuario, acción, entidad afectada
- Añadir hooks de auditoría en `articles.service.ts` y `contacts.service.ts` para registrar `create/update/delete`

T13 | Corregir configuración del Application ID de la app móvil

PROBLEMA: El `build.gradle.kts` de Flutter tiene un TODO pendiente. Si se publica sin resolver, el Application ID genérico puede colisionar en la Play Store.

- Editar: `apps/mobile/android/app/build.gradle.kts`
- Cambiar `applicationId` = `"com.acontplus.erp.mobile"` al ID real del proyecto
- Editar: `apps/mobile/android/app/src/main/AndroidManifest.xml` — verificar `android:label`
- Editar: `apps/mobile/web/manifest.json` — actualizar `name` y `short_name` desde 'mobile' al nombre real
- Editar: `apps/mobile/README.md` — reemplazar el README genérico de Flutter por documentación real del proyecto

Resumen de Tareas

#	Prioridad	Tarea	Área principal
T1	● P0 — Crítico	Conectar Dashboard con datos reales	Backend + Frontend
T2	● P0 — Crítico	Corregir inconsistencia search_vector	Base de Datos + Backend
T3	● P0 — Crítico	Transacciones en operaciones de barcodes	Backend (Service)
T4	● P1 — Alto	Implementar RoleGuard en endpoints sensibles	Backend (Guards)
T5	● P1 — Alto	Implementar flujo completo del POS	Backend + Frontend
T6	● P1 — Alto	Implementar Refresh Token en autenticación	Backend + Frontend
T7	● P2 — Medio	Extraer lógica de precios de article-form-dialog	Frontend (Refactor)
T8	● P2 — Medio	Centralizar API_BASE y crear servicios API	Frontend (Config)
T9	● P2 — Medio	Refactorizar layout del dashboard	Frontend (Componentes)
T10	○ P3 — Bajo	Añadir pruebas automatizadas a lógica crítica	Backend + Frontend
T11	○ P3 — Bajo	Configurar Swagger / OpenAPI	Backend (Docs)
T12	○ P3 — Bajo	Implementar página de Auditoría	Backend + Frontend
T13	○ P3 — Bajo	Corregir Application ID de la app móvil	Mobile (Flutter)